VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Data Structures(23CS3PCDST)

Submitted by

Sarthak Thrishuleshwar (1BM24CS260)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2025 to Jan-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Data Structures(23CS3PCDST)" carried out by **Sarthak Thrishuleshwar (1BM24CS260)**, who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025. The Lab report has been approved as it satisfies the academic requirements in respect of a Data Structures (23CS3PCDST) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Table Of Contents

Sl. No.	Experiment Title	Page No.
1	Stack Operations	1
2	Conversion of infix to postfix	4
3a	Linear Queue Operations	7
3b	Circular Queue Operations	11
4	Singly Linked List Insertion Operations	15
5	Singly Linked List Deletion Operations	22
6a	Singly Linked List Additional Operations	29
6b	Singly Linked List to Implement Stack and Queue Operations	35
7a	Doubly Linked List Operations	40
8a	Binary Search Tree Operations	45
9a	BFS Method	49
9b	DFS Method	51
10	Hash Table Operations	53
11	Leet Code Practice	56

Course outcomes:

CO1	Apply the concept of linear and nonlinear data structures.
CO2	Analyse data structure operations for a given problem
CO3	Design and develop solutions using the operations of linear and nonlinear data structure for a given specification.
CO4	Conduct practical experiments for demonstrating the operations of different data structures.

Lab program 1:

Write a program to simulate the working of stack using an array with the following:
a) Push
b) Pop
c) Display
The program should print appropriate messages for stack overflow, stack underflow.

```
#include <stdio.h>

#define MAX 5


int stack[MAX];

int top = -1;


void push(int x) {

    if (top == MAX - 1) {

        printf("Stack Overflow\n");

    } else {

        top++;

        stack[top] = x;

        printf("Pushed %d\n", x);

    }

}


void pop() {

    if (top == -1) {

        printf("Stack Underflow\n");

    } else {

        printf("Popped %d\n", stack[top]);

        top--;

    }

}
```

```
void display() {
    if (top == -1) {
        printf("Stack is Empty\n");
    } else {
        int i;
        printf("Stack elements:\n");
        for (i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    int choice, value;
    while (1) {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf ("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
```

```
            break;
        case 4:
            return 0;
        default:
            printf("Invalid Choice\n");
    }
  }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\1.exe"
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 10
Pushed 10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 20
Pushed 20

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 30
Pushed 30

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements:
30
20
10

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped 30

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements:
20
10

1. Push
2. Pop
```

Lab Program – 2

WAP to convert a given valid parenthesized infix arithmetic expression to postfix expression. The expression consists of single character operands and the binary operators + (plus), - (minus), * (multiply) and / (divide).

```
#include <stdio.h>

#include <ctype.h>


#define MAX 100


char stack[MAX];

int top = -1;


void push(char x) {

    stack[++top] = x;

}


char pop() {

    return stack[top--];

}


int precedence(char x) {

    if (x == '+' || x == '-') return 1;

    if (x == '*' || x == '/') return 2;

    return 0;

}


int main() {
```

```
    char infix[MAX], postfix[MAX];
    int i, j = 0;
    char ch;
    scanf("%s", infix);
    for (i = 0; infix[i] != '\0'; i++) {
        ch = infix[i];
        if (isalnum(ch))
            postfix[j++] = ch;
        else if (ch == '(')
            push(ch);
        else if (ch == ')') {
            while (top != -1 && stack[top] != '(')
                postfix[j++] = pop();
            pop();
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(ch))
                postfix[j++] = pop();
            push(ch);
        }
    }
    while (top != -1)
        postfix[j++] = pop();
    postfix[j] = '\0';
    printf("%s", postfix);
    return 0;
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\2a.exe"
(A+B)*(C+D)/(E-F)
AB+CD+*EF-/
Process returned 0 (0x0)   execution time : 2.567 s
Press any key to continue.
```

Lab Proram – 3

a) WAP to simulate the working of a queue of integers using an array. Provide the following operations: Insert, Delete, Display The program should print appropriate messages for queue empty and queue overflow conditions

```
#include <stdio.h>

#define MAX 5


int queue[MAX];

int front = -1, rear = -1;


void insert(int x) {

   if (rear == MAX - 1) {

      printf("Queue Overflow\n");

      return;

   }

   if (front == -1)

      front = 0;

   queue[++rear] = x;

   printf("Inserted %d\n", x);

}


void delete() {

   if (front == -1 || front > rear) {

      printf("Queue Empty\n");

      return;

   }

   printf("Deleted %d\n", queue[front++]);

   if (front > rear)

      front = rear = -1;

}


void display() {
```

```
    if (front == -1) {
        printf("Queue Empty\n");
        return;
    }
    int i;
    printf("Queue elements:\n");
    for (i = front; i <= rear; i++)
        printf("%d\n", queue[i]);
}

int main() {
    int choice, value;
    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insert(value);
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
```

```
        default:
            printf("Invalid Choice\n");
    }
  }
}
```

Output

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 10
Inserted 10

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 20
Inserted 20

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 30
Inserted 30

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue elements:
10
20
30
```

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue elements:
20
30

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 4

Process returned 0 (0x0)   execution time : 27.124 s
Press any key to continue.
```

Lab Program – 3

b) WAP to simulate the working of a circular queue of integers using an array. Provide the following operations: Insert, Delete & Display The program should print appropriate messages for queue empty and queue overflow conditions

```
#include <stdio.h>

#define MAX 5


int queue[MAX];

int front = -1, rear = -1;


void insert(int x) {

   if ((front == 0 && rear == MAX - 1) || (rear + 1 == front)) {

      printf("Queue Overflow\n");

      return;

   }

   if (front == -1) {

      front = rear = 0;

   } else {

      rear = (rear + 1) % MAX;

   }

   queue[rear] = x;

   printf("Inserted %d\n", x);

}


void delete() {

   if (front == -1) {

      printf("Queue Empty\n");

      return;

   }

   printf("Deleted %d\n", queue[front]);

   if (front == rear) {

      front = rear = -1;
```

```
    } else {
        front = (front + 1) % MAX;
    }
}

void display() {
    if (front == -1) {
        printf("Queue Empty\n");
        return;
    }
    printf("Queue elements:\n");
    int i = front;
    while (1) {
        printf("%d\n", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
}

int main() {
    int choice, value;
    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
```

```
            insert(value);
            break;
        case 2:
            delete();
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid Choice\n");
    }
  }
}
```

Output

"E:\Sarthak\Data Structures\Lab Programs\3b.exe"

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 10
Inserted 10

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 20
Invalid Choice

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 20
Inserted 20

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 30
Inserted 30
```

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue elements:
10
20
30

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
Deleted 10

1. Insert
2. Delete
3. Display
4. Exit
```

Lab Program – 4

WAP to Implement Singly Linked List with following operations a) Create a linked list. b) Insertion of a node at first position, at any position and at end of list. Display the contents of the linked list.

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>


struct node {

   int info;

   struct node *next;

};


struct node *createsllbeg()

{

   int item;

   struct node *p,*start;

   start = NULL;

   printf("Enter the item to insert into the linked list (Enter -1 to exit)\n");

   scanf("%d",&item);


   while(item!=-1)

   {

      p=(struct node *)malloc(sizeof(struct node));

      p->info=item;

      if(start==NULL)

      {

         p->next=NULL;

         start=p;

         printf("First element inserted into the linked list\n");

      }

      else
```

```
        {
            p->next=start;
            start=p;
            printf(" element inserted into the linked list\n");
        }
        scanf("%d",&item);
    }

    return start;
}

void displaysll(struct node *start)
{
    struct node *temp;
    if(start==NULL)
    {
        printf("The sll is empty\n");
        return;
    }
    else
    {
        printf("The items in the sll are as follows:\n");
        temp=start;
        while(temp!=NULL)
        {
            printf("%d\t",temp->info);
            temp=temp->next;
        }
    }
}
```

```
struct node *insertsllbeg(struct node *start,int item)
{
   struct node *p;
   p=(struct node *)malloc(sizeof(struct node));
   p->info=item;
   if(start==NULL)
   {
      p->next=NULL;
      start=p;
   }
   else
   {
      p->next=start;
      start=p;
   }
   return start;
}


struct node *insertsllend(struct node *start, int item)
{
   struct node *p,*temp;
   p=(struct node *)malloc(sizeof(struct node));
   p->info=item;
   if(start==NULL)
   {
      p->next=NULL;
      start=p;
   }
```

```
    else
    {
        temp=start;
        while(temp->next!=NULL)
        {
            temp=temp->next;
        }
        temp->next=p;
        p->next=NULL;
    }
    return start;
}


struct node* insert_at_position(struct node *start, int value, int pos) {
    int i;
    struct node *temp, *p;

    temp = (struct node*)malloc(sizeof(struct node));
    temp->info = value;
    temp->next = NULL;

    if (pos == 1) {
        temp->next = start;
        start = temp;
        return start;
    }

    p = start;
    for (i = 1; i < pos - 1 && p != NULL; i++)
```

```
        p = p->next;

    if (p == NULL)
        return start;

    temp->next = p->next;
    p->next = temp;

    return start;
}




void main()
{
    struct node *start;
    start=NULL;
    int choice,item,pos;
    printf("-----MENU FOR SINGLY LINKED LIST OPERATIONS-----\n");
    printf("1)Create sll\n");
    printf("2)Insert at beg of sll\n");
    printf("3)Insert at end of sll\n");
    printf("4)Insert at any position in sll\n");
    printf("5)Display sll\n");
    printf("6)Exit\n");
    while(1)
    {
        printf("\nEnter your choice:\n");
        scanf("%d",&choice);
        switch(choice)
        {
```

```
        case 1:
            start=createsllbeg();
            break;
        case 2:
            printf("Enter the item to insert at beg of sll:\n");
            scanf("%d",&item);
            start=insertsllbeg(start,item);
            break;
        case 3:
            printf("Enter the item to insert at end of sll:\n");
            scanf("%d",&item);
            start=insertsllend(start,item);
            break;
        case 4:
            printf("Enter the item to insert:\n");
            int item;
            scanf("%d",&item);
            printf("Enter the position to insert\n");
            scanf("%d",&pos);
            start=insert_at_position(start,item,pos);
            break;
        case 5:
            displaysll(start);
            break;
        case 6:
            exit(0);
        default:
            printf("Invalid choice! Please try again.\n");
    }
  }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\4a.exe"
-----MENU FOR SINGLY LINKED LIST OPERATIONS-----
1)Create sll
2)Insert at beg of sll
3)Insert at end of sll
4)Insert at any position in sll
5)Display sll
6)Exit

Enter your choice:
1
Enter the item to insert into the linked list (Enter -1 to exit)
10
First element inserted into the linked list
20
 element inserted into the linked list
30
 element inserted into the linked list
40
 element inserted into the linked list
50
 element inserted into the linked list
-1

Enter your choice:
2
Enter the item to insert at beg of sll:
0

Enter your choice:
3
Enter the item to insert at end of sll:
60

Enter your choice:
4
Enter the item to insert:
5
Enter the position to insert
2

Enter your choice:
5
The items in the sll are as follows:
0       5       50      40      30      20      10      60
Enter your choice:
6

Process returned 0 (0x0)   execution time : 65.286 s
Press any key to continue.
```

Lab Program – 5

WAP to Implement Singly Linked List with following operations a) Create a linked list. b) Deletion of first element, specified element and last element in the list. c) Display the contents of the linked list.

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>


struct node {

   int info;

   struct node *next;

};


struct node *createsllbeg()

{

   int item;

   struct node *p,*start, *current;

   start = NULL;

   printf("Enter the item to insert into the linked list (Enter -1 to exit)\n");

   scanf("%d",&item);


   while(item!=-1)

   {

      p=(struct node *)malloc(sizeof(struct node));

      p->info=item;

      if(start==NULL)

      {

         p->next=NULL;

         start=p;

         printf("First element inserted into the linked list\n");

      }

      else
```

```
        {
            p->next=start;
            start=p;
            printf(" element inserted into the linked list\n");
        }
        scanf("%d",&item);
    }

    return start;
}

void displaysll(struct node *start)
{
    struct node *temp;
    if(start==NULL)
    {
        printf("The sll is empty\n");
        return;
    }
    else
    {
        printf("The items in the sll are as follows:\n");
        temp=start;
        while(temp!=NULL)
        {
            printf("%d\t",temp->info);
            temp=temp->next;
        }
    }
}
```

```
struct node *deletesllbeg(struct node *start)
{
   struct node *temp;
   if(start==NULL)
   {
      printf("the sll is empty\n");
      return start;
   }
   else if(start->next==NULL)
   {
      temp=start;
      printf("Element deleted: %d\n",start->info);
      start=NULL;
      free(temp);
      return start;
   }
   else
   {
      temp=start;
      printf("Element deleted:%d",start->info);
      start=start->next;
      free(temp);
      return start;
   }
}

struct node *deletesllend(struct node *start)
{
   struct node *temp,*follow;
```

```
    if(start==NULL)
    {
        printf("The sll is empty\n");
    }
    else
    {
        temp=start;
        follow=NULL;
        while(temp->next!=NULL)
        {
            follow=temp;
            temp=temp->next;
        }
        printf("the element deleted: %d\n",temp->info);
        free(temp);
        follow->next=NULL;
    }
    return start;

}



struct node* delete_at_position(struct node *start, int pos) {
    struct node *temp, *p;
    int i;

    if (start == NULL)
        return start;

    if (pos == 1) {
        temp = start;
```

```
        start = start->next;
        free(temp);
        return start;
    }
    p = start;
    for (i = 1; i < pos - 1 && p != NULL; i++)
        p = p->next;
    if (p == NULL || p->next == NULL)
        return start;
    temp = p->next;
    p->next = temp->next;
    free(temp);
    return start;
}

void main()
{
    struct node *start;
    start=NULL;
    int choice,item,pos;
    printf("-----MENU FOR SINGLY LINKED LIST OPERATIONS-----\n");
    printf("1)Create sll\n");
    printf("2)Delete at beg of sll\n");
    printf("3)Delete at end of sll\n");
    printf("4)Delete at any pos of sll\n");
    printf("5)Display sll\n");
    printf("6)Exit\n");
    while(1)
    {
        printf("\nEnter your choice:\n");
        scanf("%d",&choice);
```

```
    switch(choice)
    {
      case 1:
        start=createsllbeg();
        break;
      case 2:
        start=deletesllbeg(start);
        break;
      case 3:
        start=deletesllend(start);
        break;
      case 4:
        printf("Enter the position to delete the element\n");
        scanf("%d",&pos);
        start=delete_at_position(start,pos);
      case 5:
        displaysll(start);
        break;
      case 6:
        exit(0);
      default:
        printf("Invalid choice! Please try again.\n");
    }
  }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\5a.exe"
-----MENU FOR SINGLY LINKED LIST OPERATIONS-----
1)Create sll
2)Delete at beg of sll
3)Delete at end of sll
4)Delete at any pos of sll
5)Display sll
6)Exit
```

```
Enter your choice:
1
Enter the item to insert into the linked list (Enter -1 to exit)
55
First element inserted into the linked list
50
 element inserted into the linked list
45
 element inserted into the linked list
40
 element inserted into the linked list
30
 element inserted into the linked list
20
 element inserted into the linked list
10
 element inserted into the linked list
0
 element inserted into the linked list
-1

Enter your choice:
5
The items in the sll are as follows:
0       10      20      30      40      45      50      55
Enter your choice:
2
Element deleted:0
Enter your choice:
3
the element deleted: 55

Enter your choice:
4
Enter the position to delete the element
5
The items in the sll are as follows:
10      20      30      40      50
Enter your choice:
6
```

"E:\Sarthak\Data Structures\Lab Programs\5a.exe"

```
1)Create sll
2)Delete at beg of sll
3)Delete at end of sll
4)Delete at any pos of sll
5)Display sll
6)Exit

Enter your choice:
1
Enter the item to insert into the linked list (Enter -1 to exit)
10
First element inserted into the linked list
-1

Enter your choice:
2
Element deleted: 10

Enter your choice:
3
The sll is empty

Enter your choice:
4
Enter the position to delete the element
5
The sll is empty
```

Lab Program – 6

a) WAP to Implement Single Link List with following operations: Sort the linked list, Reverse the linked list, Concatenation of two linked lists.

```
#include<stdio.h>

#include<stdlib.h>


struct node {

   int info;

   struct node *next;

};


// ---------------- CREATE LIST ----------------

struct node* create_list() {

   int item;

   struct node *start = NULL, *temp;


   printf("Enter elements (-1 to stop): ");

   scanf("%d", &item);


   while (item != -1) {

      temp = (struct node*)malloc(sizeof(struct node));

      temp->info = item;

      temp->next = start;

      start = temp;

      scanf("%d", &item);

   }

   return start;

}


// ---------------- DISPLAY LIST ----------------

void display(struct node *start) {
```

```
    struct node *temp = start;
    if (start == NULL) {
        printf("List empty\n");
        return;
    }
    while (temp != NULL) {
        printf("%d ", temp->info);
        temp = temp->next;
    }
    printf("\n");
}

// ---------------- SORT LIST ----------------
struct node* sort_list(struct node *start) {
    struct node *temp, *follow;
    int t;

    if (start == NULL)
        return start;

    for (temp = start; temp->next != NULL; temp = temp->next) {
        for (follow = temp->next; follow != NULL; follow = follow->next) {
            if (temp->info > follow->info) {
                t = temp->info;
                temp->info = follow->info;
                follow->info = t;
            }
        }
    }
    return start;
}
```

```
// ---------------- REVERSE LIST ----------------
struct node* reverse_list(struct node *start) {
   struct node *temp = NULL, *follow = NULL;

   while (start != NULL) {
      follow = start->next;
      start->next = temp;
      temp = start;
      start = follow;
   }
   return temp;
}

// ---------------- CONCATENATE LISTS ----------------
struct node* concatenate(struct node *start1, struct node *start2) {
   struct node *temp;

   if (start1 == NULL)
      return start2;
   if (start2 == NULL)
      return start1;

   temp = start1;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = start2;
   return start1;
}
```

```
// ---------------- MAIN MENU ----------------
int main() {

    struct node *start1 = NULL, *start2 = NULL;
    int choice;

    while (1) {
        printf("\n--- MENU ---\n");
        printf("1. Create 1st List\n");
        printf("2. Create 2nd List\n");
        printf("3. Display Lists\n");
        printf("4. Sort 1st List\n");
        printf("5. Reverse 1st List\n");
        printf("6. Concatenate List2 to List1\n");
        printf("7. Exit\n");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                start1 = create_list();
                break;
            case 2:
                start2 = create_list();
                break;
            case 3:
                printf("List 1: ");
                display(start1);
                printf("List 2: ");
                display(start2);
                break;
```

```
        case 4:
            start1 = sort_list(start1);
            break;
        case 5:
            start1 = reverse_list(start1);
            break;
        case 6:
            start1 = concatenate(start1, start2);
            printf("Lists concatenated.\n");
            break;
        case 7:
            exit(0);
    }
  }
  return 0;
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\6a.exe"

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
1
Enter elements (-1 to stop): 20
30
10
-1

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
2
Enter elements (-1 to stop): 60
50
40
-1

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
3
List 1: 10 30 20
List 2: 40 50 60
```

"E:\Sarthak\Data Structures\Lab Programs\6a.exe"

```
--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
4

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
3
List 1: 10 20 30
List 2: 40 50 60

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
5

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
3
List 1: 30 20 10
List 2: 40 50 60
```

```
--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
6
Lists concatenated.

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
3
List 1: 30 20 10 40 50 60
List 2: 40 50 60

--- MENU ---
1. Create 1st List
2. Create 2nd List
3. Display Lists
4. Sort 1st List
5. Reverse 1st List
6. Concatenate List2 to List1
7. Exit
7

Process returned 0 (0x0)   execution time : 141.369 s
Press any key to continue.
```

Lab Program – 6

b) WAP to Implement Single Link List to simulate Stack & Queue Operations.

```
#include<stdio.h>
#include<stdlib.h>

struct node {
   int info;
   struct node *next;
};

struct node* push(struct node *start, int item) {
   struct node *p = (struct node*)malloc(sizeof(struct node));
   p->info = item;
   p->next = start;
   start = p;
   return start;
}

struct node* pop(struct node *start) {
   struct node *temp;
   if (start == NULL) return start;
   temp = start;
   start = start->next;
   free(temp);
   return start;
}

void display(struct node *start) {
   struct node *temp = start;
   if (start == NULL) {
      printf("Empty\n");
```

```
        return;
    }
    while (temp != NULL) {
        printf("%d ", temp->info);
        temp = temp->next;
    }
    printf("\n");
}

struct node* enqueue(struct node *start, int item) {
    struct node *p, *temp;
    p = (struct node*)malloc(sizeof(struct node));
    p->info = item;
    p->next = NULL;

    if (start == NULL) {
        start = p;
        return start;
    }

    temp = start;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = p;
    return start;
}

struct node* dequeue(struct node *start) {
    struct node *temp;
    if (start == NULL) return start;
```

```
    temp = start;
    start = start->next;
    free(temp);
    return start;
}

int main() {
    struct node *stack = NULL, *queue = NULL;
    int ch, item;

    while (1) {
        printf("\n1 Push\n2 Pop\n3 Display Stack\n4 Enqueue\n5 Dequeue\n6 Display Queue\n7
Exit\n");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                scanf("%d",&item);
                stack = push(stack, item);
                break;
            case 2:
                stack = pop(stack);
                break;
            case 3:
                display(stack);
                break;
            case 4:
                scanf("%d",&item);
                queue = enqueue(queue, item);
                break;
            case 5:
```

```
            queue = dequeue(queue);
            break;
        case 6:
            display(queue);
            break;
        case 7:
            exit(0);
    }
  }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\6b.exe"

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
2
The stack is empty

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
1
30

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
1
20

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
3
20 30

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
2
```

"E:\Sarthak\Data Structures\Lab Programs\6b.exe"

```
1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
3
30

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
5
The queue is empty

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
4
10

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
4
20

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
6
10 20
```

```
1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
5

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
6
10 20

1 Push
2 Pop
3 Display Stack
4 Enqueue
5 Dequeue
6 Display Queue
7 Exit
7

Process returned 0 (0x0)   execution time : 40.732 s
Press any key to continue.
```

Lab Program 7

WAP to Implement doubly link list with primitive operations a) Create a doubly linked list. b) Insert a new node to the left of the node. c) Delete the node based on a specific value d) Display the contents of the list

```
#include<stdio.h>

#include<stdlib.h>


struct node {

   int info;

   struct node *next;

   struct node *prev;

};


struct node *createdllbeg()

{

   struct node *p,*start=NULL;

   int item;

   printf("Enter the item to insert\n");

   scanf("%d",&item);

   while(item!=-1)

   {

      p=(struct node *)malloc(sizeof(struct node));

      p->info=item;

      p->next=NULL;

      p->prev=NULL;

      if(start==NULL)

      {

         start=p;

      }

      else

      {

         p->next=start;
```

```
            start->prev=p;
            start=p;
        }
        scanf("%d",&item);
    }
    return start;
}

void displaydll(struct node *start)
{
    struct node *temp;
    if(start==NULL)
    {
        printf("The dll is empty\n");

    }
    else
    {
        temp=start;
        while(temp!=NULL)
        {
            printf("%d\t",temp->info);
            temp=temp->next;
        }
        printf("\n");
    }
}

struct node *insertdll(struct node *start, int item, int key)
{
    struct node *p,*temp;
```

```
    p=(struct node *)malloc(sizeof(struct node));
    p->info=item;
    temp=start;
    while(temp!=NULL && temp->info!=key)
        temp=temp->next;
    if(temp==start)
    {
        p->prev=NULL;
        p->next=start;
        start->prev=p;
        start=p;
    }
    else
    {
        p->prev=temp->prev;
        p->next=temp;
        temp->prev->next=p;
        temp->prev=p;
    }
    return start;
}

struct node *deleteval(struct node *start, int item)
{
    struct node *temp,*follow;
    temp=start;
    while(temp!=NULL && temp->info!=item)
    {
        follow=temp;
        temp=temp->next;
    }
```

```
   if(temp==NULL)
   {
      printf("Item not found\n");
      return start;
   }
   if(temp==start)
   {
      start=start->next;
      free(temp);
   }
   else
   {
      follow->next=temp->next;
      temp->next->prev=follow;
      free(temp);
   }
   return start;
}

void main()
{
  int ch,item,key;
  struct node *start=NULL;
  while(1)
  {
     printf("1)Create\n2)Insert\n3)Delete\n4)Display\n5)Exit\nEnter your choice:\n");
     scanf("%d",&ch);
     switch(ch)
     {
        case 1:  start=createdllbeg();
                 break;
```

```
        case 2:  printf("Enter the item and key\n");
                 scanf("%d%d",&item,&key);
                 start=insertdll(start,item,key);
                 break;
        case 3:  printf("Enter the value to delete\n");
                 scanf("%d",&item);
                 start=deleteval(start,item);
                 break;
        case 4:  displaydll(start);
                 break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
      }
  }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\7a.exe"
1)Create
2)Insert
3)Delete
4)Display
5)Exit
Enter your choice:
1
Enter the item to insert
50
40
30
20
10
-1
1)Create
2)Insert
3)Delete
4)Display
5)Exit
Enter your choice:
2
Enter the item and key
45
50
1)Create
2)Insert
3)Delete
4)Display
5)Exit
Enter your choice:
4
10      20      30      40      45      50
1)Create
2)Insert
3)Delete
4)Display
5)Exit
Enter your choice:
3
Enter the value to delete
45
1)Create
2)Insert
3)Delete
4)Display
5)Exit
Enter your choice:
4
10      20      30      40      50
```

Lab Program – 8

Write a program a) To construct a binary Search tree. b) To traverse the tree using all the methods i.e., in-order, preorder and post order c) To display the elements in the tree.

```
#include <stdio.h>

#include <stdlib.h>


struct node {

   int info;

   struct node *left;

   struct node *right;

};


struct node* insert(struct node *root, int item) {

   struct node *p;

   if (root == NULL) {

      p = (struct node*)malloc(sizeof(struct node));

      p->info = item;

      p->left = NULL;

      p->right = NULL;

      return p;

   }

   if (item < root->info)

      root->left = insert(root->left, item);

   else if (item > root->info)

      root->right = insert(root->right, item);

   return root;

}


void inorder(struct node *root) {

   if (root != NULL) {

      inorder(root->left);
```

```
        printf("%d ", root->info);
        inorder(root->right);
    }
}

void preorder(struct node *root) {
    if (root != NULL) {
        printf("%d ", root->info);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->info);
    }
}

int main() {
    struct node *root = NULL;
    int choice, item;
    while (1) {
        printf("\n1.Insert\n2.Inorder\n3.Preorder\n4.Postorder\n5.Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                scanf("%d", &item);
                root = insert(root, item);
```

```
            break;
        case 2:
            inorder(root);
            break;
        case 3:
            preorder(root);
            break;
        case 4:
            postorder(root);
            break;
        case 5:
            exit(0);
      }
   }
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\8a.exe"
1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
100

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
70

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
120

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
60

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
80

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
110
```

"E:\Sarthak\Data Structures\Lab Programs\8a.exe"

```
1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
1
200

1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
2
60 70 80 100 110 120 200
1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
3
100 70 60 80 120 110 200
1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
4
60 80 70 110 200 120 100
1.Insert
2.Inorder
3.Preorder
4.Postorder
5.Exit
```

Lab Program – 9

a) Write a program to traverse a graph using BFS method.

```
#include <stdio.h>

int n;
int adj[20][20];
int visited[20];
int queue[20];
int front = -1, rear = -1;

void bfs(int start) {
   int i, v;

   visited[start] = 1;
   queue[++rear] = start;

   printf("BFS Traversal: ");

   while (front != rear) {
      v = queue[++front];
      printf("%d ", v);

      for (i = 0; i < n; i++) {
         if (adj[v][i] == 1 && visited[i] == 0) {
            visited[i] = 1;
            queue[++rear] = i;
         }
      }
   }
}
```

```
int main() {
    int i, j, start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for (i = 0; i < n; i++)
        visited[i] = 0;

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    bfs(start);

    return 0;
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\9a.exe"
Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 0
1 0 1 0
0 1 0 1
0 0 1 0
Enter starting vertex: 0
BFS Traversal: 0 1 2 3
```

```
"E:\Sarthak\Data Structures\Lab Programs\9a.exe"
Enter number of vertices: 1
Enter adjacency matrix:
0
Enter starting vertex: 0
BFS Traversal: 0
```

Lab Program – 9

b) Write a program to check whether given graph is connected or not using DFS method.

```
#include <stdio.h>

int n;
int adj[20][20];
int visited[20];

void dfs(int v) {
   int i;
   visited[v] = 1;

   for (i = 0; i < n; i++) {
      if (adj[v][i] == 1 && visited[i] == 0) {
         dfs(i);
      }
   }
}

int main() {
   int i, j;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &adj[i][j]);

   for (i = 0; i < n; i++)
```

```
        visited[i] = 0;

    dfs(0);   // start DFS from vertex 0

    for (i = 0; i < n; i++) {
        if (visited[i] == 0) {
            printf("Graph is NOT Connected\n");
            return 0;
        }
    }

    printf("Graph is Connected\n");
    return 0;
}
```

Output

```
"E:\Sarthak\Data Structures\Lab Programs\9b.exe"
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Graph is Connected
```

```
"E:\Sarthak\Data Structures\Lab Programs\9b.exe"
Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 0
1 0 0 0
0 0 0 1
0 0 1 0
Graph is NOT Connected
```

Lab Program 10

Given a File of N employee records with a set K of Keys(4 digit) which uniquely determine the records in file F. 10 5 10 Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are integers. Design and develop a Program in C that uses Hash function H: K -> L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.

```
#include <stdio.h>


int ht[100], m;


void insert(int key)
{
   int index = key % m;
   int start = index;


   while (ht[index] != -1)
   {
      index = (index + 1) % m;
      if (index == start)
      {
         printf("Hash table is full\n");
         return;
      }
   }
   ht[index] = key;
}


void display()
{
```

```
    int i;
    for (i = 0; i < m; i++)
        printf("%d : %d\n", i, ht[i]);
}

int main()
{
    int n, key, i;

    printf("Enter size of hash table: ");
    scanf("%d", &m);

    for (i = 0; i < m; i++)
        ht[i] = -1;

    printf("Enter number of keys: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter key: ");
        scanf("%d", &key);
        insert(key);
    }

    display();
    return 0;
}
```

Output

```
Enter size of hash table: 3
Enter number of keys: 3
Enter key: 10
Enter key: 20
Enter key: 30
0 : 30
1 : 10
2 : 20
```

```
Enter size of hash table: 3
Enter number of keys: 2
Enter key: 10
Enter key: 20
0 : -1
1 : 10
2 : 20
```

```
Enter size of hash table: 3
Enter number of keys: 4
Enter key: 3
Enter key: 4
Enter key: 5
Enter key: 5
Hash table is full
0 : 3
1 : 4
2 : 5
```

LEET Code Practice Problems

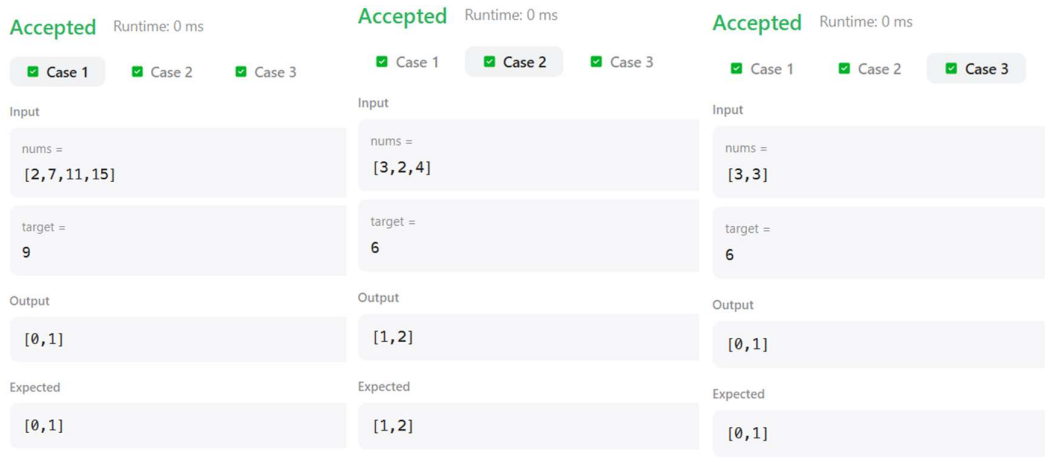
2b) Two Sum

```
int* twoSum(int* nums, int numsSize, int target, int* returnSize)
{
    static int result[2];

    for (int i = 0; i < numsSize; i++)
    {
        for (int j = i + 1; j < numsSize; j++)
        {
            if (nums[i] + nums[j] == target)
            {
                result[0] = i;
                result[1] = j;
                *returnSize = 2;
                return result;
            }
        }
    }

    *returnSize = 0;
    return NULL;
}
```

Output



4b) Remove Linked List Elements

```
struct ListNode* removeElements(struct ListNode* head, int val) {
    struct ListNode *temp = head, *follow = NULL;

    while (temp != NULL) {
        if (temp->val == val) {
            if (follow == NULL) {
                head = temp->next;
```

```
                free(temp);
                temp = head;
            } else {
                follow->next = temp->next;
                free(temp);
                temp = follow->next;
            }
        } else {
            follow = temp;
            temp = temp->next;
        }
    }
    return head;
}
```

Output

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[1,2,6,3,4,5,6]

val =
6

Output

[1,2,3,4,5]

Expected

[1,2,3,4,5]

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[]

val =
1

Output

[]

Expected

[]

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[7,7,7,7]

val =
7

Output

[]

Expected

[]

7b) Linked List Cycle

```
bool hasCycle(struct ListNode *head) {
    struct ListNode *slow = head;
    struct ListNode *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
            return true;
    }
    return false;
}
```

Output

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[3,2,0,-4]

pos =
1

Output

true

Expected

true

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[1,2]

pos =
0

Output

true

Expected

true

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =
[1]

pos =
-1

Output

false

Expected

false

8b) Merge Two Binary Trees

```
struct TreeNode* mergeTrees(struct TreeNode* root1, struct TreeNode* root2) {
    if (root1 == NULL)
        return root2;
    if (root2 == NULL)
        return root1;

    root1->val = root1->val + root2->val;
    root1->left = mergeTrees(root1->left, root2->left);
    root1->right = mergeTrees(root1->right, root2->right);

    return root1;
}
```

Output

Accepted Runtime: 0 ms

Case 1 Case 2

Input

root1 =
[1,3,2,5]

root2 =
[2,1,3,null,4,null,7]

Output

[3,4,5,5,4,null,7]

Expected

[3,4,5,5,4,null,7]

Accepted Runtime: 0 ms

Case 1 Case 2

Input

root1 =
[1]

root2 =
[1,2]

Output

[2,2]

Expected

[2,2]